APEX / PURPLLE TECH CHALLENGE 2026 - ROUND 2

Store Intelligence

Turning raw CCTV footage into live, queryable store analytics.

> North Star: offline conversion rate = visitors who bought / unique visitors in a window.

> End-to-end: raw video -> detection -> event stream -> intelligence API -> live dashboard.

> Built for imperfect, real-world footage: groups, staff, re-entry, occlusion, empty periods.

1. The Problem

> A specialty retail chain has mature ONLINE analytics but its physical stores are a data blind spot.

> Goal: reconstruct the offline funnel from CCTV alone - how many came in, where they went, who bought.

> Every design choice either makes the conversion number MORE ACCURATE (detection) or MORE USEFUL (API).

> No pre-processed data, no skeleton code - every decision from frame to API response is ours to make.

2. System Architecture

Two independent processes joined only by an event schema.

- Process 1 - Detection pipeline (heavy, offline, GPU): video -> YOLO + ByteTrack -> geometry/staff/dedup -> events.jsonl.
- Process 2 - Intelligence API (lightweight, NO ML deps): ingest -> metrics / funnel / heatmap / anomalies / health.
- Why split: the graded API image builds in seconds, so 'docker compose up' (the gate) never breaks on a CV dependency.
- Plus two operator tools: a browser Calibration Studio (draw zones) and a live Dashboard.

3. Detection Pipeline (Part A, 30 pts)

> YOLO11m (COCO person) + ByteTrack - chosen for occlusion recall, not speed (the pipeline is offline).

> Entry counting is SINGLE-CAMERA via a tripwire: removes cross-camera double-counting by construction.

> Zones = hand-drawn polygons on the distorted frame (no lens calibration); point-in-polygon tests.

> Staff = cascade: position oracle -> Gemini VLM behavioural confirmer -> persistence heuristic.

> Appearance dedup (torchvision MobileNetV3 embeddings) collapses one person's many per-camera tokens.

> Output: schema-validated events.jsonl (event_id, visitor_id, event_type, timestamp, zone, is_staff, confidence).

4. Edge Cases Handled

The footage includes the same edge cases as a real deployment.

- Group entry -> count individuals, not groups (one ENTRY per person at the tripwire).
- Staff movement -> flagged is_staff and excluded from every customer metric.
- Re-entry -> the same person returning emits REENTRY, not a second ENTRY (no visitor inflation).
- Partial occlusion -> low-confidence boxes kept (ByteTrack 2nd association) and flagged, never silently dropped.
- Empty-store periods -> API returns clean zeros, never null/NaN, never crashes.
- Camera overlap -> embedding dedup so the same person is not double-counted across angles.

5. Event Schema + Adapter

- Canonical flat StoreEvent row; event_id is the primary key AND the idempotency key.
- Two additive metadata keys (id_source, id_confidence) surface weak identity instead of hiding it.
- A normalization adapter ingests BOTH the original schema AND the newer multi-shape stream

  (entry / zone_entered / queue_completed with demographics + groups) - folding all into one row.
- Store-id normalization (store_1076 <-> ST1076) and deterministic event_ids keep ingest idempotent.

6. Intelligence API (Part B, 35 pts)

- POST /events/ingest - batch <=500, per-event validation, idempotent, partial success.
- GET /metrics - unique visitors, conversion, dwell-by-zone, queue depth, abandonment, demographics, groups.
- GET /funnel - ENTRY -> ZONE_VISIT -> BILLING_QUEUE -> PURCHASE, deduped on the visitor (re-entries counted once).
- GET /heatmap, GET /anomalies (queue spike / conversion drop / dead zone), GET /health (stale-feed).
- Conversion without identity: a visitor in the billing zone within 5 min before a POS txn = converted.
- All query-time, store-agnostic, staff-excluded, explicit zeros on zero traffic.

7. Production Readiness (Part C, 20 pts)

- Containerised: 'docker compose up' starts Postgres + API + dashboard, no manual steps beyond git clone.
- Structured JSON logs per request: trace_id, store_id, endpoint, latency_ms, event_count, status_code.
- Idempotent ingest (ON CONFLICT DO NOTHING) and DB-unavailable -> HTTP 503 with no stack traces.
- 69 tests, ~85% coverage, on an ephemeral SQLite DB (same upsert path as Postgres) - no server needed.
- Dual-dialect SQLAlchemy Core: Postgres in prod, SQLite in tests; indexes defined for scale.

8. AI Engineering (Part D, 15 pts)

Evaluated for HOW we used AI - depth and intentionality, not volume.

> Model choice: AI suggested 'lightest for speed'; we OVERRODE it - offline means recall-under-occlusion wins.

> Staff: we first overrode the VLM, then ADOPTED it on real footage (floor consultant), then made it conservative (confidence gate + customer-biased prompt) after honest evaluation showed it over-flagged.

> AI-assisted debugging: found OSNet/torchreid silently dies on torch 2.x -> switched to MobileNet -> dedup finally works.

> Every test file carries a # PROMPT / # CHANGES MADE block; DESIGN.md + CHOICES.md document the overrides.

9. Calibration Studio + Live Dashboard (Part E)

- Studio (browser): pick a camera, EXTRACT FRAMES from its video, draw zone polygons / tripwires, see them overlaid.

  No OpenCV display needed; frames are extracted at native resolution so drawings align with the pipeline.
- Writes straight into each store's zones.json / lines.json - what you draw is what the pipeline runs.
- Dashboard: polls /metrics, /funnel, /anomalies every 2s; visitor + conversion counters update as events replay.
- Proof the pipeline and API are genuinely connected, not batch-faked.

10. How to Run

- Gate (API only): docker compose up -d --build -> GET /stores/STORE_BLR_002/metrics.
- Pipeline (per store): docker compose -f docker-compose.pipeline.yml run --rm -e OUTPUT_PATH=... -e MANIFEST=... pipeline.
- Merge events -> replay: dashboard/replay.py --events data/events.jsonl --api http://localhost:8000 --speed 20.
- Tune with env vars: VID_STRIDE, REID_DEDUP_THRESHOLD, VLM_STAFF_CONF, VLM_MIN_INTERVAL_S.
- Full step-by-step (put videos -> extract frames -> draw zones -> run) + troubleshooting is in README.md.

11. Honest Limitations & Next Steps

> Cross-camera identity is best-effort (blurred faces, similar clothing) - flagged, never gates the headline count.

> The funnel is an aggregate unique-count funnel, not a per-individual trace.

> Conversion needs a POS feed whose store_id matches the footage; a mismatched feed correctly reports 0.

> Next: per-camera homography for true cross-camera linking; per-camera dedup thresholds; demographic funnel splits.

> At 40 live stores the GPU-bound detection layer saturates first; the query-time API + Postgres scale comfortably.